

 MAUÁ	PREENCHA OS CAMPOS EM AMARELO		
Disc.: ECM251 – LINGUAGEM DE PROGRAMAÇÃO I			
Curso: ENGENHARIA DA COMPUTAÇÃO			

Aluno: <COLOQUE SEU NOME AQUI>	Test Code:
Curso: ECM Série: 3º Período: MATUTINO RG:	<CÓDIGO>
São Caetano do Sul, 22 de setembro de 2026.	RA: <SEU RA>
Assinatura:	Nota:

Instruções da Prova

- Esta prova é individual e prática, devendo ser realizada, **obrigatoriamente**, nos computadores instalados nos laboratórios designados do IMT;
 - Não poderá haver acesso à *Internet*, sob nenhuma circunstância, exceto ao *CanvasLMS* do próprio aluno e, mesmo assim, somente em duas etapas: para receber (“baixar”) as questões de sua prova na máquina em uso; e para entregar (“subir”) as resoluções das questões de sua prova no devido local de entrega, na mesma plataforma;
 - Poderá haver consulta a qualquer material do próprio aluno, seja ele físico (livros, artigos, material de aula etc.) ou digital (livros, artigos, material de aula, exercícios, resoluções etc.), desde que esse material seja seu e esteja previamente armazenado em um dispositivo de armazenamento externo (*pendrive*, *drive* externo etc.).
 - O aluno deverá responder às questões da prova gerando os códigos necessários, somente na linguagem de programação **JAVA** e utilizando sempre e somente o paradigma da Programação Orientada a Objetos e os comandos estudados nas aulas da disciplina;
 - Para realizar a entrega da prova na plataforma CanvasLMS, o aluno deverá gerar e entregar um único arquivo compactado (*.zip*), tendo seu RA e seu NOME completo como nome desse arquivo. Nesse arquivo compactado o aluno deverá fornecer, obrigatoriamente, os seus dados acadêmicos preenchidos na folha de prova (*.docx*) e os arquivos e códigos gerados em suas resoluções, uma pasta para cada questão, contendo as suas classes e demais arquivos que possibilitem sua posterior execução pelo professor, durante a resolução.
- IMPORTANTE:** Na correção, para executar o código gerado pelo aluno em sua prova, o professor seguirá exatamente as instruções fornecidas pelos alunos e contidas nas resoluções das provas! Caso não obtenha sucesso, a questão será considerada errada.
- Não poderá haver troca de informações, nem de materiais, sejam físicos ou digitais, entre os alunos durante a prova;
 - Não é permitido ao estudante se ausentar da sala antes da entrega da prova;
 - Celulares, *smartphones*, *smartwatches*, *smartglasses* ou outros equipamentos eletrônicos devem permanecer desligados e fora do alcance físico e/ou visual do aluno enquanto ele estiver na sala;
 - O tempo limite para realização da prova é de **90** minutos;
 - Mantenha sobre a carteira apenas um documento com foto, caneta, lápis, borracha e seu material para consulta;
 - O entendimento das questões faz parte da avaliação;
 - O tempo mínimo de permanência na sala é de **30** minutos;
 - O estudante que chegar atrasado, depois dos **30** minutos do início da prova, **não** poderá fazê-la!

QUESTÃO ÚNICA – REGISTRO SEGURO DE OCORRÊNCIA

Fornecer o código-fonte, em *Java*, das classes necessárias e o arquivo executável *.jar*, implementadas sob o paradigma da Programação Orientada a Objetos, incluindo suas interfaces com o usuário geradas automaticamente pelo *NetBeans*, utilizando apenas os comandos apresentados em aula, para resolver o seguinte problema:

História

Um laboratório precisa registrar ocorrências como falhas em equipamentos, indisponibilidade de *software* e problemas de infraestrutura. Como os relatos podem conter informações sensíveis, eles devem ser armazenados de forma cifrada.

Desenvolva uma aplicação *Java* denominada **Registro Seguro de Ocorrência**, atendendo aos itens a seguir.

A. Interface gráfica obrigatoriamente criada no *NetBeans*

Crie um projeto *Java with Ant* e construa, obrigatoriamente com um *JFrame Form* do *NetBeans*, uma interface contendo:

- Campo para o nome do responsável;
- Campo protegido para a senha;
- Seleção da gravidade: Baixa, Média ou Alta;
- Área para a descrição da ocorrência;
- Data e hora atuais;
- Seleção do idioma: Português (Brasil) ou Inglês (EUA);
- Botões Registrar, Consultar e Sair.

A interface deve ser organizada, legível e funcional e deve ser aderente à figura abaixo:

A imagem mostra uma janela de aplicativo intitulada "Registro Seguro de Ocorrências". No topo, há um ícone de cadeado e o título da janela. À direita, há um menu suspenso para "Idioma:" com "Português (Brasil)" selecionado. O formulário principal contém os seguintes campos e controles:

- Responsável:** Um campo de texto para o nome do responsável.
- Senha:** Um campo de texto com pontos para ocultar o conteúdo.
- Gravidade:** Um menu suspenso com a opção "Selecione..." visível.
- Data e hora:** Um campo de texto exibindo "15/09/2026 09:30:00".
- Descrição da ocorrência:** Uma área de texto grande para a descrição detalhada.

Na base da janela, há três botões: "Registrar" (em azul), "Consultar" e "Sair".

Atenção: A geração automática de código pelo *NetBeans* será permitida exclusivamente para a construção e inicialização visual do formulário. A lógica dos eventos e as demais classes deverão ser programadas **manualmente** pelo aluno, em qualquer IDE.

B. Senha, data e ícone

A senha deverá ser capturada por um *JPasswordField*. Uma ocorrência somente poderá ser registrada quando:

- O responsável estiver preenchido;
- A descrição possuir pelo menos 10 caracteres;
- Uma gravidade tiver sido selecionada;
- A senha digitada for **ECM251**.

A data e a hora do registro deverão ser obtidas no momento da operação e apresentadas no formato:

dd/MM/yyyy HH:mm:ss

C. Internacionalização

A interface deverá funcionar em:

- Português do Brasil;
- Inglês dos Estados Unidos.

Crie arquivos *.properties* contendo as mesmas chaves nos dois idiomas. A alteração do idioma deverá atualizar, durante a execução:

- Títulos e rótulos;
- Textos dos botões;
- Opções de gravidade.

Os textos internacionalizados não poderão permanecer fixos no código *Java*.

D. Arquivo-texto

Os dados da ocorrência deverão ser gravados no arquivo:

ocorrencias.txt

Somente a última ocorrência deverá estar gravada e deverá conter:

- Data e hora;
- Responsável;
- Gravidade;
- Descrição cifrada.

O botão **Consultar** deverá ler o arquivo e apresentar o registro anteriormente gravado na mesma GUI. O programa deverá tratar adequadamente a inexistência inicial do arquivo.

E. Criptografia

A descrição da ocorrência deverá ser cifrada com **AES**, podendo ser utilizada a classe **CryptoAES.java** fornecida nas aulas.

A aplicação deverá:

1. Cifrar a descrição antes de gravá-la;
2. Impedir que o texto original apareça em *ocorrencias.txt*;
3. Decifrar o conteúdo durante a consulta;
4. Reutilizar a chave simétrica já existente nas execuções posteriores.

F. Orientação a objetos

Separe, no mínimo, as seguintes responsabilidades:

- Representação de uma ocorrência;
- Validação das regras;
- Acesso ao arquivo-texto;
- Criptografia e descriptografia;
- Interface gráfica.

Não concentre todas as operações na classe do **JFrame**. A interface deverá chamar objetos responsáveis pelas regras e pela persistência.

Entrega

Entregue um único arquivo compactado contendo:

- Projeto completo;
- Códigos-fonte;
- Formulários do *NetBeans*;
- Arquivos *.properties*;
- Ícone;
- Arquivo *.jar* executável;
- Classe *CryptoAES.java*;
- Arquivo da chave criptográfica;
- Instruções breves para executar a aplicação.

O arquivo compactado deverá ser identificado com o *RA* e o *nome do aluno*.

Rubrica de avaliação

Critério	Valor	Evidências esperadas
Interface construída no NetBeans	2,0	<i>JFrame Form</i> organizado, componentes solicitados e eventos funcionais
Senha, data, ícone e validações	1,5	<i>JPasswordField</i> , data formatada, visualização coerente do ícone e mensagens adequadas
Internacionalização	1,5	<i>Locale</i> , <i>ResourceBundle</i> , dois arquivos <i>.properties</i> e atualização efetiva da interface
Leitura e gravação em arquivo-texto	1,5	Persistência, recuperação do registro e tratamento do arquivo inexistente
Criptografia AES	1,5	Cifra antes da gravação, decifra na consulta e reutilização correta da chave
Orientação a objetos e integração	1,0	Responsabilidades separadas e ausência de lógica excessiva no <i>JFrame</i>
Executável e entrega completa	1,0	<i>.jar</i> funcional, dependências, recursos e instruções incluídos
Total	10,00	

A pontuação parcial deverá ser atribuída quando o item estiver implementado de maneira incompleta, mas tecnicamente identificável e executável.

Condições eliminatórias

A prova poderá receber nota zero quando:

- Não for implementada em *Java*;
- A interface não for construída em um *JFrame Form* do *NetBeans*;
- O projeto entregue não possuir código-fonte;
- Houver uso de bibliotecas, *frameworks* ou recursos não autorizados;
- Houver código gerado por inteligência artificial ou outra ferramenta proibida;
- O programa não puder ser executado por falta de arquivos ou instruções essenciais.

A geração visual efetuada pelo **GUI Builder** do *NetBeans* constitui uma exceção expressamente autorizada.